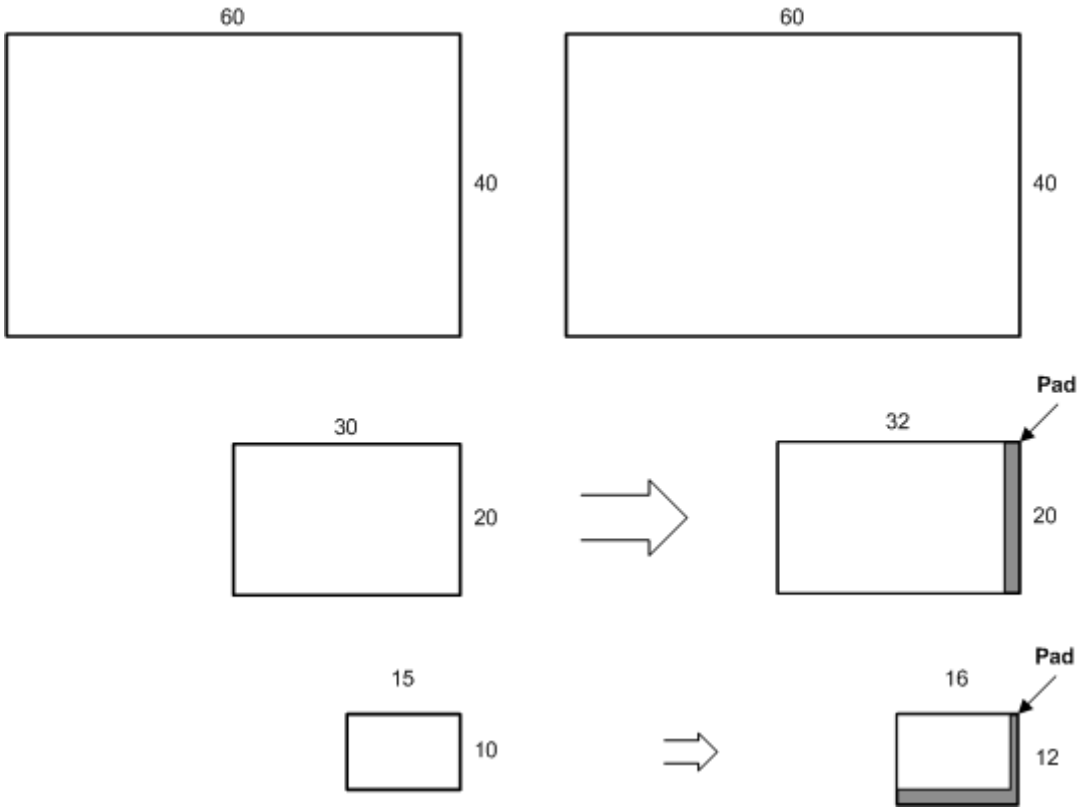
Use a block compressed texture the same way you would use an uncompressed texture. If your application will get a memory pointer to block-compressed data, you need to account for the memory padding in a mipmap that causes the declared size to differ from the actual size.

- Virtual size versus physical size

Virtual size versus physical size

If you have application code that uses a memory pointer to walk the memory of a block compressed texture, there is one important consideration that may require a modification in your application code. A block-compressed texture must be a multiple of 4 in all dimensions because the block-compression algorithms operate on 4x4 texel blocks. This will be a problem for a mipmap whose initial dimensions are divisible by 4, but subdivided levels are not. The following diagram shows the difference in area between the virtual (declared) size and the physical (actual) size of each mipmap level.



The left side of the diagram shows the mipmap level sizes that are generated for an uncompressed 60×40 texture. The top level size is taken from the API call that generates the texture; each subsequent level is half the size of the previous level. For an uncompressed texture, there is no difference between the virtual (declared) size and the physical (actual) size.

The right side of the diagram shows the mipmap level sizes that are generated for the same 60×40 texture with compression. Notice that both the second and third levels

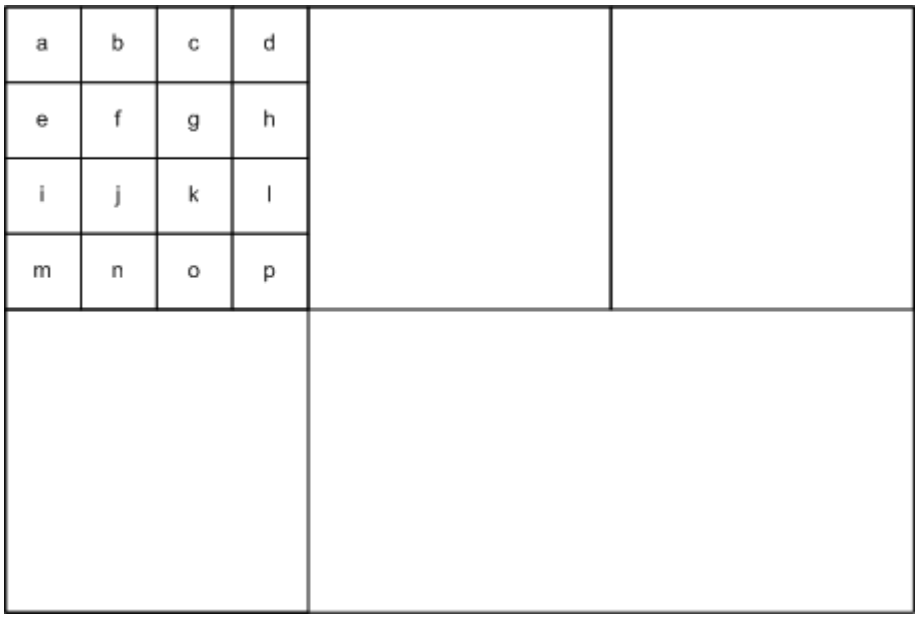
have memory padding to make the sizes factors of 4 on every level. This is necessary so that the algorithms can operate on 4×4 texel blocks. This is especially evident if you consider mipmap levels that are smaller than 4×4; the size of these very small mipmap levels will be rounded up to the nearest factor of 4 when texture memory is allocated.

Sampling hardware uses the virtual size; when the texture is sampled, the memory padding is ignored. For mipmap levels that are smaller than 4×4, only the first four texels will be used for a 2×2 map, and only the first texel will be used by a 1×1 block. However, there is no API structure that exposes the physical size (including the memory padding).

In summary, be careful to use aligned memory blocks when copying regions that contain block-compressed data. To do this in an application that gets a memory pointer, make sure that the pointer uses the surface pitch to account for the physical memory size.

Compression algorithms

Block compression techniques in Direct3D break up uncompressed texture data into 4×4 blocks, compress each block, and then store the data. For this reason, textures are expected to be compressed must have texture dimensions that are multiples of 4.



The preceding diagram shows a texture partitioned into texel blocks. The first block shows the layout of the 16 texels labeled a-p, but every block has the same organization of data.

Direct3D implements several compression schemes, each implements a different tradeoff between number of components stored, the number of bits per component,

and the amount of memory consumed. Use this table to help choose the format that works best with the type of data and the data resolution that best fits your application.

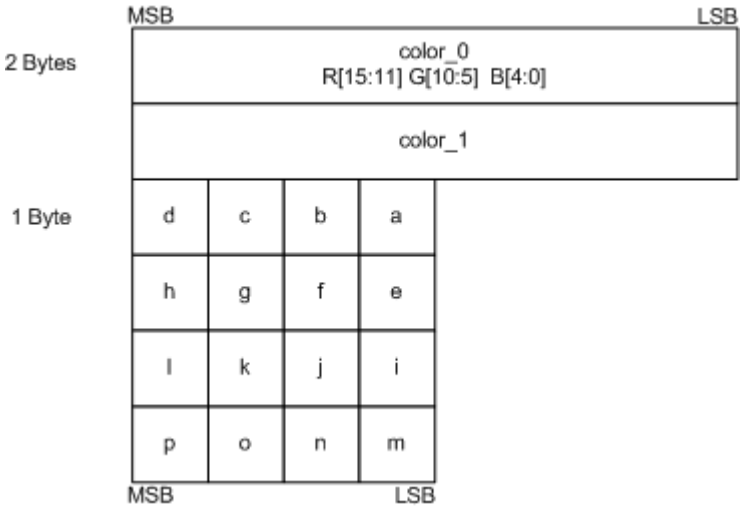
Source Data	Data Compression Resolution (in bits)	Choose this compression format
Three-component color and alpha	Color (5:6:5), Alpha (1) or no alpha	BC1
Three-component color and alpha	Color (5:6:5), Alpha (4)	BC2
Three-component color and alpha	Color (5:6:5), Alpha (8)	BC3
One-component color	One component (8)	BC4
Two-component color	Two components (8:8)	BC5

- BC1
- BC2
- BC3
- BC4
- BC5

BC1

Use the first block compression format (BC1) (either DXGI_FORMAT_BC1_TYPELESS, DXGI_FORMAT_BC1_UNORM, or DXGI_BC1_UNORM_SRGB) to store three-component color data using a 5:6:5 color (5 bits red, 6 bits green, 5 bits blue). This is true even if the data also contains 1-bit alpha. Assuming a 4×4 texture using the largest data format possible, the BC1 format reduces the memory required from 48 bytes (16 colors × 3 components/color × 1 byte/component) to 8 bytes of memory.

The algorithm works on 4×4 blocks of texels. Instead of storing 16 colors, the algorithm saves 2 reference colors (color_0 and color_1) and 16 2-bit color indices (blocks a–p), as shown in the following diagram.



The color indices (a–p) are used to look up the original colors from a color table. The color table contains 4 colors. The first two colors—color_0 and color_1—are the minimum and maximum colors. The other two colors, color_2 and color_3, are intermediate colors calculated with linear interpolation.

```
C++

color_2 = 2/3*color_0 + 1/3*color_1
color_3 = 1/3*color_0 + 2/3*color_1
```

The four colors are assigned 2-bit index values that will be saved in blocks a–p.

```
C++

color_0 = 00
color_1 = 01
color_2 = 10
color_3 = 11
```

Finally, each of the colors in blocks a–p are compared with the four colors in the color table, and the index for the closest color is stored in the 2-bit blocks.

This algorithm lends itself to data that contains 1-bit alpha also. The only difference is that color_3 is set to 0 (which represents a transparent color) and color_2 is a linear blend of color_0 and color_1.

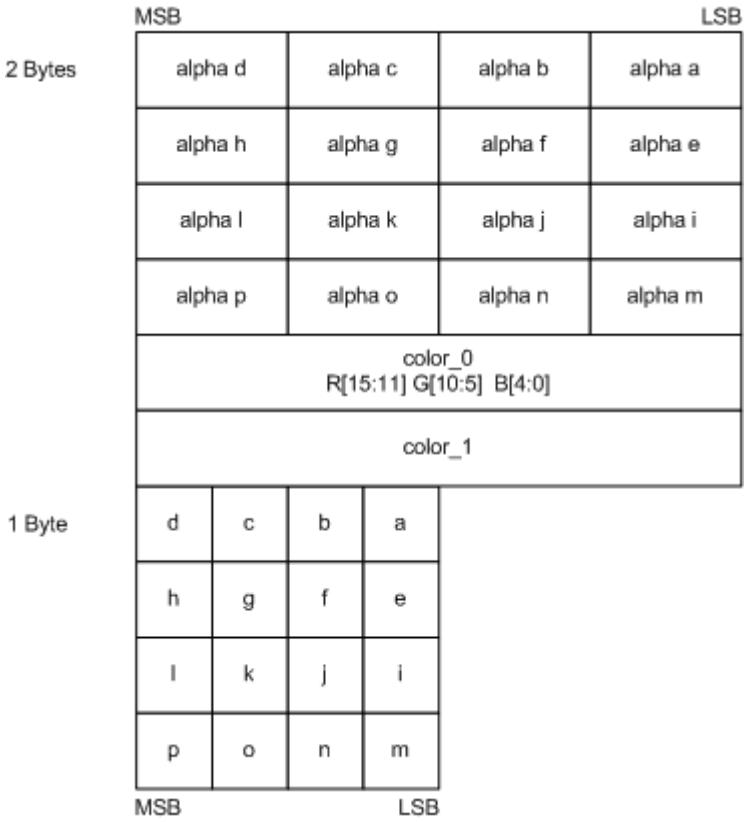
```
C++

color_2 = 1/2*color_0 + 1/2*color_1;
color_3 = 0;
```

BC2

Use the BC2 format (either DXGI_FORMAT_BC2_TYPELESS, DXGI_FORMAT_BC2_UNORM, or DXGI_BC2_UNORM_SRGB) to store data that contains color and alpha data with low coherency (use BC3 for highly-coherent alpha data). The BC2 format stores RGB data as a 5:6:5 color (5 bits red, 6 bits green, 5 bits blue) and alpha as a separate 4-bit value. Assuming a 4×4 texture using the largest data format possible, this compression technique reduces the memory required from 64 bytes (16 colors × 4 components/color × 1 byte/component) to 16 bytes of memory.

The BC2 format stores colors with the same number of bits and data layout as the BC1 format; however, BC2 requires an additional 64-bits of memory to store the alpha data, as shown in the following diagram.



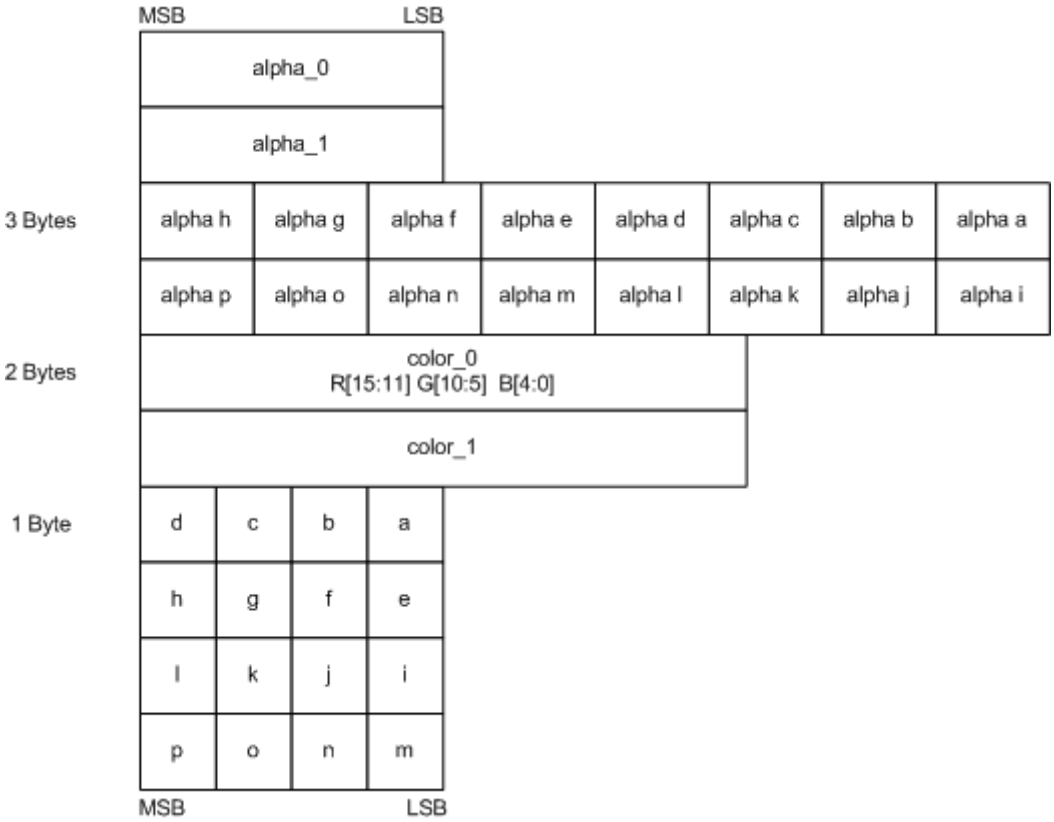
BC3

Use the BC3 format (either DXGI_FORMAT_BC3_TYPELESS, DXGI_FORMAT_BC3_UNORM, or DXGI_BC3_UNORM_SRGB) to store highly coherent color data (use BC2 with less coherent alpha data). The BC3 format stores color data using 5:6:5 color (5 bits red, 6 bits green, 5 bits blue) and alpha data using one byte. Assuming a 4×4 texture using the largest data format possible, this compression technique reduces the memory required from 64 bytes (16 colors × 4 components/color × 1 byte/component) to 16 bytes of memory.

The BC3 format stores colors with the same number of bits and data layout as the BC1 format; however, BC3 requires an additional 64-bits of memory to store the alpha data.

The BC3 format handles alpha by storing two reference values and interpolating between them (similarly to how BC1 stores RGB color).

The algorithm works on 4×4 blocks of texels. Instead of storing 16 alpha values, the algorithm stores 2 reference alphas (alpha_0 and alpha_1) and 16 3-bit color indices (alpha a through p), as shown in the following diagram.



The BC3 format uses the alpha indices (a–p) to look up the original colors from a lookup table that contains 8 values. The first two values—alpha_0 and alpha_1—are the minimum and maximum values; the other six intermediate values are calculated using linear interpolation.

The algorithm determines the number of interpolated alpha values by examining the two reference alpha values. If alpha_0 is greater than alpha_1, then BC3 interpolates 6 alpha values; otherwise, it interpolates 4. When BC3 interpolates only 4 alpha values, it sets two additional alpha values (0 for fully transparent and 255 for fully opaque). BC3 compresses the alpha values in the 4×4 texel area by storing the bit code corresponding to the interpolated alpha values which most closely matches the original alpha for a given texel.

C++

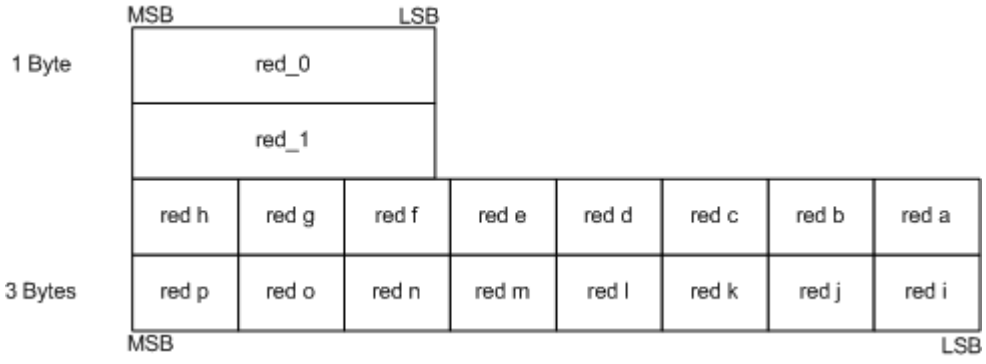
```
if( alpha_0 > alpha_1 )
{
  // 6 interpolated alpha values.
  alpha_2 = 6/7*alpha_0 + 1/7*alpha_1; // bit code 010
```

```
    alpha_3 = 5/7*alpha_0 + 2/7*alpha_1; // bit code 011
    alpha_4 = 4/7*alpha_0 + 3/7*alpha_1; // bit code 100
    alpha_5 = 3/7*alpha_0 + 4/7*alpha_1; // bit code 101
    alpha_6 = 2/7*alpha_0 + 5/7*alpha_1; // bit code 110
    alpha_7 = 1/7*alpha_0 + 6/7*alpha_1; // bit code 111
  }
  else
  {
    // 4 interpolated alpha values.
    alpha_2 = 4/5*alpha_0 + 1/5*alpha_1; // bit code 010
    alpha_3 = 3/5*alpha_0 + 2/5*alpha_1; // bit code 011
    alpha_4 = 2/5*alpha_0 + 3/5*alpha_1; // bit code 100
    alpha_5 = 1/5*alpha_0 + 4/5*alpha_1; // bit code 101
    alpha_6 = 0;                         // bit code 110
    alpha_7 = 255;                       // bit code 111
  }
```

BC4

Use the BC4 format to store one-component color data using 8 bits for each color. As a result of the increased accuracy (compared to BC1), BC4 is ideal for storing floating-point data in the range of [0 to 1] using the DXGI_FORMAT_BC4_UNORM format and [-1 to +1] using the DXGI_FORMAT_BC4_SNORM format. Assuming a 4×4 texture using the largest data format possible, this compression technique reduces the memory required from 16 bytes (16 colors × 1 components/color × 1 byte/component) to 8 bytes.

The algorithm works on 4×4 blocks of texels. Instead of storing 16 colors, the algorithm stores 2 reference colors (red_0 and red_1) and 16 3-bit color indices (red a through red p), as shown in the following diagram.



The algorithm uses the 3-bit indices to look up colors from a color table that contains 8 colors. The first two colors—red_0 and red_1—are the minimum and maximum colors. The algorithm calculates the remaining colors using linear interpolation.

The algorithm determines the number of interpolated color values by examining the two reference values. If red_0 is greater than red_1, then BC4 interpolates 6 color values; otherwise, it interpolates 4. When BC4 interpolates only 4 color values, it sets two

additional color values (0.0f for fully transparent and 1.0f for fully opaque). BC4 compresses the alpha values in the 4×4 texel area by storing the bit code corresponding to the interpolated alpha values that most closely matches the original alpha for a given texel.

- BC4_UNORM
- BC4_SNORM

BC4_UNORM

The interpolation of the single-component data is done as in the following code sample.

```
C++
```

```
unsigned word red_0, red_1;

if( red_0 > red_1 )
{
  // 6 interpolated color values
  red_2 = (6*red_0 + 1*red_1)/7.0f; // bit code 010
  red_3 = (5*red_0 + 2*red_1)/7.0f; // bit code 011
  red_4 = (4*red_0 + 3*red_1)/7.0f; // bit code 100
  red_5 = (3*red_0 + 4*red_1)/7.0f; // bit code 101
  red_6 = (2*red_0 + 5*red_1)/7.0f; // bit code 110
  red_7 = (1*red_0 + 6*red_1)/7.0f; // bit code 111
}
else
{
  // 4 interpolated color values
  red_2 = (4*red_0 + 1*red_1)/5.0f; // bit code 010
  red_3 = (3*red_0 + 2*red_1)/5.0f; // bit code 011
  red_4 = (2*red_0 + 3*red_1)/5.0f; // bit code 100
  red_5 = (1*red_0 + 4*red_1)/5.0f; // bit code 101
  red_6 = 0.0f;                     // bit code 110
  red_7 = 1.0f;                     // bit code 111
}
```

The reference colors are assigned 3-bit indices (000–111 since there are 8 values), which will be saved in blocks red a through red p during compression.

BC4_SNORM

The DXGI_FORMAT_BC4_SNORM is exactly the same, except that the data is encoded in SNORM range and when 4 color values are interpolated. The interpolation of the single-component data is done as in the following code sample.

```
C++
```

```
signed word red_0, red_1;

if( red_0 > red_1 )
{
  // 6 interpolated color values
  red_2 = (6*red_0 + 1*red_1)/7.0f; // bit code 010
  red_3 = (5*red_0 + 2*red_1)/7.0f; // bit code 011
  red_4 = (4*red_0 + 3*red_1)/7.0f; // bit code 100
  red_5 = (3*red_0 + 4*red_1)/7.0f; // bit code 101
  red_6 = (2*red_0 + 5*red_1)/7.0f; // bit code 110
  red_7 = (1*red_0 + 6*red_1)/7.0f; // bit code 111
}
else
{
  // 4 interpolated color values
  red_2 = (4*red_0 + 1*red_1)/5.0f; // bit code 010
  red_3 = (3*red_0 + 2*red_1)/5.0f; // bit code 011
  red_4 = (2*red_0 + 3*red_1)/5.0f; // bit code 100
  red_5 = (1*red_0 + 4*red_1)/5.0f; // bit code 101
  red_6 = -1.0f;                    // bit code 110
  red_7 =  1.0f;                    // bit code 111
}
```

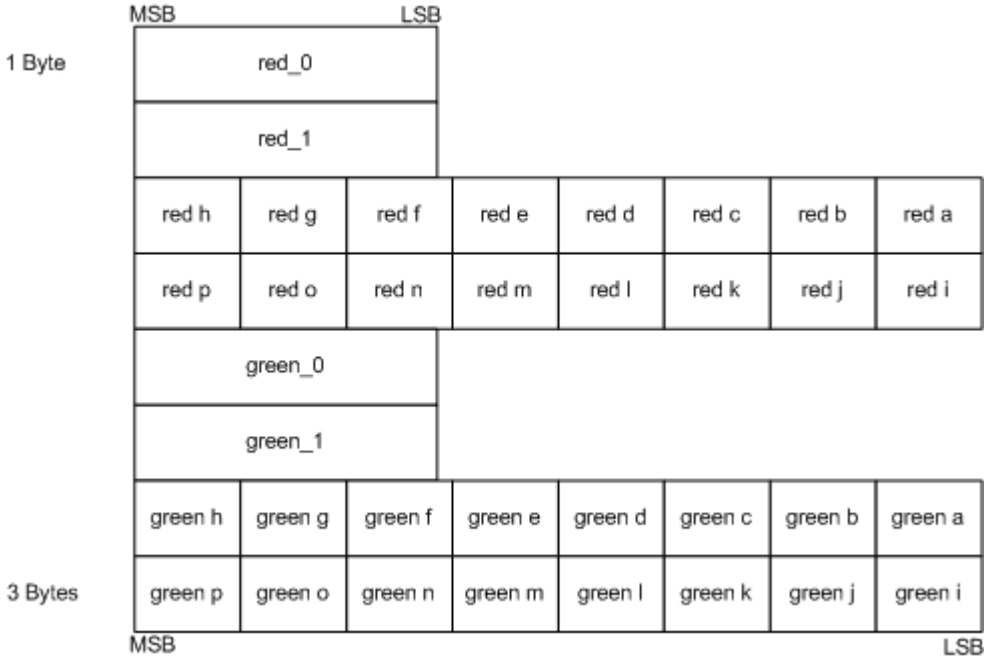
The reference colors are assigned 3-bit indices (000–111 since there are 8 values), which will be saved in blocks red a through red p during compression.

BC5

Use the BC5 format to store two-component color data using 8 bits for each color. As a result of the increased accuracy (compared to BC1), BC5 is ideal for storing floating-point data in the range of [0 to 1] using the DXGI_FORMAT_BC5_UNORM format and [-1 to +1] using the DXGI_FORMAT_BC5_SNORM format. Assuming a 4×4 texture using the largest data format possible, this compression technique reduces the memory required from 32 bytes (16 colors × 2 components/color × 1 byte/component) to 16 bytes.

- BC5_UNORM
- BC5_SNORM

The algorithm works on 4×4 blocks of texels. Instead of storing 16 colors for both components, the algorithm stores 2 reference colors for each component (red_0, red_1, green_0, and green_1) and 16 3-bit color indices for each component (red a through red p and green a through green p), as shown in the following diagram.



The algorithm uses the 3-bit indices to look up colors from a color table that contains 8 colors. The first two colors—red_0 and red_1 (or green_0 and green_1)—are the minimum and maximum colors. The algorithm calculates the remaining colors using linear interpolation.

The algorithm determines the number of interpolated color values by examining the two reference values. If red_0 is greater than red_1, then BC5 interpolates 6 color values; otherwise, it interpolates 4. When BC5 interpolates only 4 color values, it sets the remaining two color values at 0.0f and 1.0f.

BC5_UNORM

The interpolation of the single-component data is done as in the following code sample. The calculations for the green components are similar.

C++

```
unsigned word red_0, red_1;

if( red_0 > red_1 )
{
  // 6 interpolated color values
  red_2 = (6*red_0 + 1*red_1)/7.0f; // bit code 010
  red_3 = (5*red_0 + 2*red_1)/7.0f; // bit code 011
  red_4 = (4*red_0 + 3*red_1)/7.0f; // bit code 100
  red_5 = (3*red_0 + 4*red_1)/7.0f; // bit code 101
  red_6 = (2*red_0 + 5*red_1)/7.0f; // bit code 110
  red_7 = (1*red_0 + 6*red_1)/7.0f; // bit code 111
}
else
{
```

```
  // 4 interpolated color values
  red_2 = (4*red_0 + 1*red_1)/5.0f; // bit code 010
  red_3 = (3*red_0 + 2*red_1)/5.0f; // bit code 011
  red_4 = (2*red_0 + 3*red_1)/5.0f; // bit code 100
  red_5 = (1*red_0 + 4*red_1)/5.0f; // bit code 101
  red_6 = 0.0f;                     // bit code 110
  red_7 = 1.0f;                     // bit code 111
}
```

The reference colors are assigned 3-bit indices (000–111 since there are 8 values), which will be saved in blocks red a through red p during compression.

BC5_SNORM

The DXGI_FORMAT_BC5_SNORM is exactly the same, except that the data is encoded in SNORM range and when 4 data values are interpolated, the two additional values are -1.0f and 1.0f. The interpolation of the single-component data is done as in the following code sample. The calculations for the green components are similar.

C++

```
signed word red_0, red_1;

if( red_0 > red_1 )
{
  // 6 interpolated color values
  red_2 = (6*red_0 + 1*red_1)/7.0f; // bit code 010
  red_3 = (5*red_0 + 2*red_1)/7.0f; // bit code 011
  red_4 = (4*red_0 + 3*red_1)/7.0f; // bit code 100
  red_5 = (3*red_0 + 4*red_1)/7.0f; // bit code 101
  red_6 = (2*red_0 + 5*red_1)/7.0f; // bit code 110
  red_7 = (1*red_0 + 6*red_1)/7.0f; // bit code 111
}
else
{
  // 4 interpolated color values
  red_2 = (4*red_0 + 1*red_1)/5.0f; // bit code 010
  red_3 = (3*red_0 + 2*red_1)/5.0f; // bit code 011
  red_4 = (2*red_0 + 3*red_1)/5.0f; // bit code 100
  red_5 = (1*red_0 + 4*red_1)/5.0f; // bit code 101
  red_6 = -1.0f;                    // bit code 110
  red_7 =  1.0f;                    // bit code 111
}
```

The reference colors are assigned 3-bit indices (000–111 since there are 8 values), which will be saved in blocks red a through red p during compression.

Format conversion

Direct3D enables copies between prestructured-typed textures and block-compressed textures of the same bit widths.

You can copy resources between a few format types. This type of copy operation performs a type of format conversion that reinterprets resource data as a different format type. Consider this example that shows the difference between reinterpreting data with the way a more typical type of conversion behaves:

```
C++

FLOAT32 f = 1.0f;
UINT32 u;
```

To reinterpret 'f' as the type of 'u', use memcpy:

```
C++

memcpy( &u, &f, sizeof( f ) ); // 'u' becomes equal to 0x3F800000.
```

In the preceding reinterpretation, the underlying value of the data doesn't change; memcpy reinterprets the float as an unsigned integer.

To perform the more typical type of conversion, use assignment:

```
C++

u = f; // 'u' becomes 1.
```

In the preceding conversion, the underlying value of the data changes.

The following table lists the allowable source and destination formats that you can use in this reinterpretation type of format conversion. You must encode the values properly for the reinterpretation to work as expected.

Bit Width	Uncompressed Resource	Block-Compressed Resource
32	DXGI_FORMAT_R32_UINT DXGI_FORMAT_R32_SINT	DXGI_FORMAT_R9G9B9E5_SHAREDEXP

Bit Width	Uncompressed Resource	Block-Compressed Resource
64	DXGI_FORMAT_R16G16B16A16_UINT	DXGI_FORMAT_BC1_UNORM[_SRGB]
	DXGI_FORMAT_R16G16B16A16_SINT	DXGI_FORMAT_BC4_UNORM
	DXGI_FORMAT_R32G32_UINT	DXGI_FORMAT_BC4_SNORM
	DXGI_FORMAT_R32G32_SINT	
128	DXGI_FORMAT_R32G32B32A32_UINT	DXGI_FORMAT_BC2_UNORM[_SRGB]
	DXGI_FORMAT_R32G32B32A32_SINT	DXGI_FORMAT_BC3_UNORM[_SRGB]
		DXGI_FORMAT_BC5_UNORM
		DXGI_FORMAT_BC5_SNORM

Related topics

Compressed texture resources

Compressed texture formats

Article • 10/20/2022

This section contains information about the internal organization of compressed texture formats. You do not need these details to use compressed textures, because you can use Direct3D functions for conversion to and from compressed formats. However, this information is useful if you want to operate on compressed surface data directly.

Direct3D uses a compression format that divides texture maps into 4x4 texel blocks. If the texture contains no transparency - is opaque - or if the transparency is specified by a 1-bit alpha, an 8-byte block represents the texture map block. If the texture map does contain transparent texels, using an alpha channel, a 16-byte block represents it.

Any single texture must specify that its data is stored as 64 or 128 bits per group of 16 texels. If 64-bit blocks - that is, format BC1 - are used for the texture, it is possible to mix the opaque and 1-bit alpha formats on a per-block basis within the same texture. In other words, the comparison of the unsigned integer magnitude of color_0 and color_1 is performed uniquely for each block of 16 texels.

When 128-bit blocks are used, the alpha channel must be specified in either explicit (format BC2) or interpolated mode (format BC3) for the entire texture. As with color, when interpolated mode is selected, either eight interpolated alphas or six interpolated alphas mode can be used on a block-by-block basis. Again the magnitude comparison of alpha_0 and alpha_1 is done uniquely on a block-by-block basis.

The pitch for BCn formats is measured in bytes (not blocks). For example, if you have a width of 16, then you will have a pitch of four blocks (4*8 for BC1, 4*16 for BC2 or BC3).

Related topics

Compressed texture resources

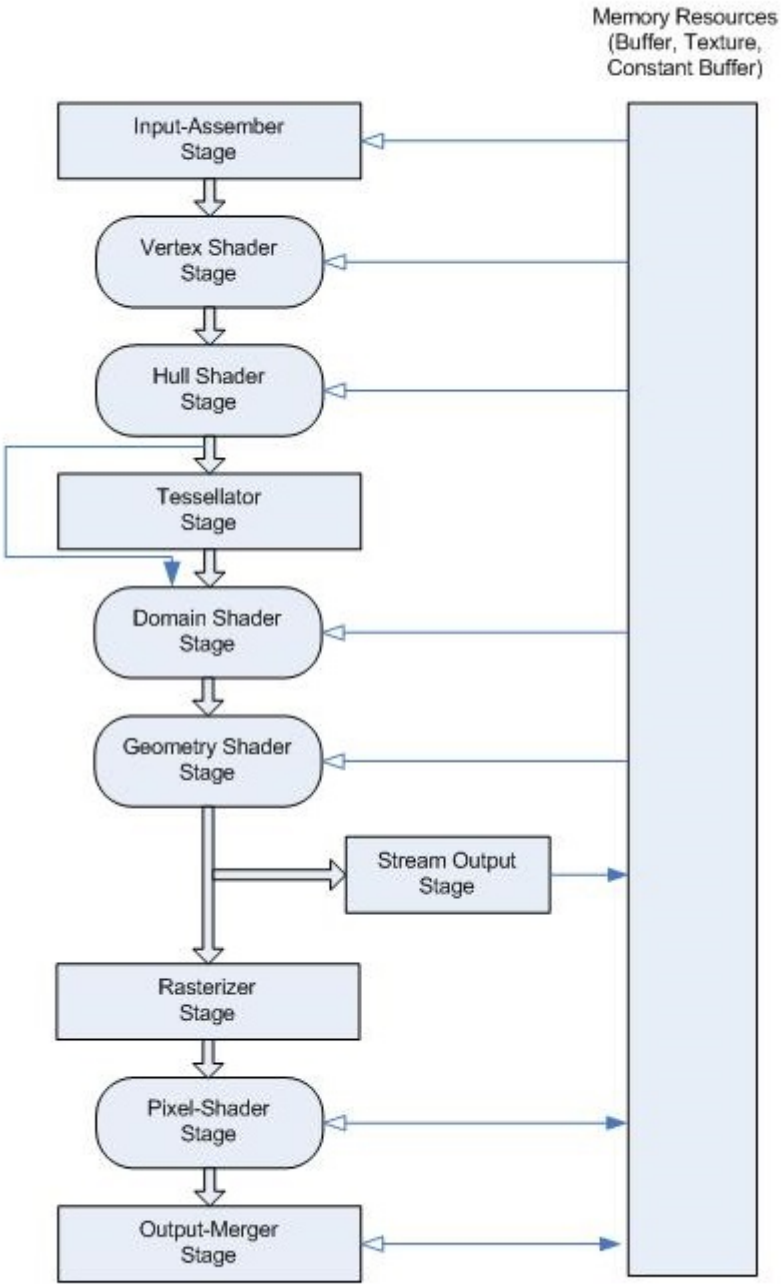
Graphics pipeline

Article • 10/20/2022

The Direct3D graphics pipeline is designed for generating graphics for realtime gaming applications. Data flows from input to output through each of the configurable or programmable stages.

All of the stages can be configured using the Direct3D API. Stages that feature common shader cores (the rounded rectangular blocks) are programmable by using the HLSL programming language. This makes the pipeline extremely flexible and adaptable.

The most commonly used are the Vertex Shader (VS) stage and the Pixel Shader (PS) stage. If you don't provide even these shader stages, a default no-op, pass-through vertex and pixel shader are used.



Input Assembler stage

The Input Assembler (IA) stage supplies primitive and adjacency data to the pipeline, such as triangles, lines and points, including semantics IDs to help make shaders more efficient by reducing processing to primitives that haven't already been processed.

- Input

  Primitive data (triangles, lines, and/or points), from user-filled buffers in memory. And possibly adjacency data. A triangle would be 3 vertices for each triangle and possibly 3 vertices for adjacency data per triangle.

- Output

  Primitives with attached system-generated values (such as a primitive ID, an instance ID, or a vertex ID).

Vertex Shader stage

The Vertex Shader (VS) stage processes vertices, typically performing operations such as transformations, skinning, and lighting. A vertex shader takes a single input vertex and produces a single output vertex. Individual per-vertex operations, such as Transformations, Skinning, Morphing and Per-vertex lighting.

- Input

  A single vertex, with VertexID and InstanceID system-generated values. Each vertex shader input vertex can be comprised of up to 16 32-bit vectors (up to 4 components each).

- Output

  A single vertex. Each output vertex can be comprised of as many as 16 32-bit 4-component vectors.

Hull Shader stage

The Hull Shader (HS) stage is one of the tessellation stages, which efficiently break up a single surface of a model into many triangles. A hull shader is invoked once per patch, and it transforms input control points that define a low-order surface into control points that make up a patch. It also does some per-patch calculations to provide data for the Tessellator (TS) stage and the Domain Shader (DS) stage.

- Input

  Between 1 and 32 input control points, which together define a low-order surface.

- Output

  Between 1 and 32 output control points, which together make up a patch. The hull shader declares the state for the Tessellator (TS) stage, including the number of control points, the type of patch face, and the type of partitioning to use when tessellating.

Tessellator stage

The Tessellator (TS) stage creates a sampling pattern of the domain that represents the geometry patch and generates a set of smaller objects (triangles, points, or lines) that connect these samples.

- Input

  The tessellator operates once per patch using the tessellation factors (which specify how finely the domain will be tessellated) and the type of partitioning (which specifies the algorithm used to slice up a patch) that are passed in from the hull-shader stage.

- Output

  The tessellator outputs uv (and optionally w) coordinates and the surface topology to the domain-shader stage.

Domain Shader stage

The Domain Shader (DS) stage calculates the vertex position of a subdivided point in the output patch; it calculates the vertex position that corresponds to each domain sample. A domain shader is run once per tessellator stage output point and has read-only access to the hull shader output patch and output patch constants, and the tessellator stage output UV coordinates.

- Input

  A domain shader consumes output control points from the Hull Shader (HS) stage. The hull shader outputs include: Control points, Patch constant data, and Tessellation factors (the tessellation factors can include the values used by the fixed-function tessellator as well as the raw values - before rounding by integer tessellation - which facilitates geomorphing, for example). A domain shader is invoked once per output coordinate from the Tessellator (TS) stage.

- Output

  The Domain Shader (DS) stage outputs the vertex position of a subdivided point in the output patch.

Geometry Shader stage

The Geometry Shader (GS) stage processes entire primitives: triangles, lines, and points, along with their adjacent vertices. It supports geometry amplification and de-amplification. It is useful for algorithms including Point Sprite Expansion, Dynamic Particle Systems, Fur/Fin Generation, Shadow Volume Generation, Single Pass Render-to-Cubemap, Per-Primitive Material Swapping, and Per-Primitive Material Setup - including generation of barycentric coordinates as primitive data so that a pixel shader can perform custom attribute interpolation.

- Input

  Unlike vertex shaders, which operate on a single vertex, the geometry shader's inputs are the vertices for a full primitive (three vertices for triangles, two vertices for lines, or single vertex for point).

- Output

  The Geometry Shader (GS) stage is capable of outputting multiple vertices forming a single selected topology. Available geometry shader output topologies are **tristrip**, **linestrip**, and **pointlist**. The number of primitives emitted can vary freely within any invocation of the geometry shader, though the maximum number of vertices that could be emitted must be declared statically. Strip lengths emitted from a geometry shader invocation can be arbitrary, and new strips can be created via the RestartStrip HLSL function.

Stream Output stage

The Stream Output (SO) stage continuously outputs (or streams) vertex data from the previous active stage to one or more buffers in memory. Data streamed out to memory can be recirculated back into the pipeline as input data, or read-back from the CPU.

- Input

  Vertex data from a previous pipeline stage.

- Output

  The Stream Output (SO) stage continuously outputs (or streams) vertex data from the previous active stage, such as the Geometry Shader (GS) stage, to one or more buffers in memory. If the Geometry Shader (GS) stage is inactive, and the Stream Output (SO) stage is active, it continuously outputs vertex data from the Domain Shader (DS) stage to buffers in memory (or if the DS is also inactive, from the Vertex Shader (VS) stage).

Rasterizer stage

The Rasterizer (RS) stage clips primitives that aren't in view, prepares primitives for the Pixel Shader (PS) stage, and determines how to invoke pixel shaders. Converts vector information (composed of shapes or primitives) into a raster image (composed of pixels) for the purpose of displaying real-time 3D graphics.

- Input

  Vertices (x,y,z,w) coming into the Rasterizer stage are assumed to be in homogeneous clip-space. In this coordinate space the X axis points right, Y points up and Z points away from camera.

- Output

  The actual pixels that need to be rendered. Includes some vertex attributes for use in interpolation by the Pixel Shader.

Pixel Shader stage

The Pixel Shader (PS) stage receives interpolated data for a primitive and generates per-pixel data such as color.

- Input

  When the pipeline is configured without a geometry shader, a pixel shader is limited to 16, 32-bit, 4-component inputs. Otherwise, a pixel shader can take up to 32, 32-bit, 4-component inputs. Pixel shader input data includes vertex attributes (that can be interpolated with or without perspective correction) or can be treated as per-primitive constants. Pixel shader inputs are interpolated from the vertex attributes of the primitive being rasterized, based on the interpolation mode declared. If a primitive gets clipped before rasterization, the interpolation mode is honored during the clipping process as well.

- Output

  A pixel shader can output up to 8, 32-bit, 4-component colors, or no color if the pixel is discarded. Pixel shader output register components must be declared before they can be used; each register is allowed a distinct output-write mask.

Output Merger stage

The Output Merger (OM) stage combines various types of output data (pixel shader values, depth and stencil information) with the contents of the render target and depth/stencil buffers to generate the final pipeline result.

- Input

  Output Merger inputs are the Pipeline state, the pixel data generated by the pixel shaders, the contents of the render targets and the contents of the depth/stencil buffers.

- Output

  The final rendered pixel color.

Related topics

- Direct3D Graphics Learning Guide
- Compute pipeline

Input Assembler (IA) stage

Article • 10/20/2022

The Input Assembler (IA) stage supplies primitive and adjacency data to the pipeline, such as triangles, lines and points, including semantics IDs to help make shaders more efficient by reducing processing to primitives that haven't already been processed.

Purpose and uses

The purpose of the Input Assembler (IA) stage is to read primitive data (points, lines and triangles) from user-filled buffers and assemble the data into primitives that will be used by the other pipeline stages, and to attach system-generated values to help make shaders more efficient. System-generated values are text strings that are also called semantics. The programmable shader stages are constructed from a common shader core, which uses system-generated values (such as a primitive ID, an instance ID, or a vertex ID), so that the shader stage can reduce processing to only those primitives, instances, or vertices that have not already been processed.

The IA stage can assemble vertices into several different primitive types (such as line lists, triangle strips, or primitives with adjacency). Primitive types such as a triangle list with adjacency, and a line list with adjacency, support the Geometry Shader (GS) stage.

Adjacency information is visible to an application only in a geometry shader. If a geometry shader were invoked with a triangle including adjacency, for instance, the input data would contain 3 vertices for each triangle and 3 vertices for adjacency data per triangle.

When the IA stage is requested to output adjacency data, the input data must include adjacency data. This may require providing a dummy vertex (forming a degenerate triangle), or perhaps by flagging in one of the vertex attributes whether the vertex exists or not. This would also need to be detected and handled by a geometry shader, although culling of degenerate geometry will happen in the rasterizer stage.

Input

The IA stage reads data from memory: primitive data (points, lines and/or triangles), from user-filled buffers.

Output

The IA stage assembles the data into primitives and attaches system-generated values, and outputs that as primitives that will be used by the Vertex Shader (VS) stage and then other pipeline stages.

In this section

Topic	Description
Primitive topologies	Direct3D supports several primitive topologies, which define how vertices are interpreted and rendered by the pipeline, such as point lists, line lists, and triangle strips.
Using system-generated values	System-generated values are generated by the Input Assembler (IA) stage (based on user-supplied input semantics) to allow certain efficiencies in shader operations. By attaching data, such as an instance ID (visible to the Vertex Shader (VS) stage), a vertex ID (visible to VS), or a primitive ID (visible to Geometry Shader (GS) stage/Pixel Shader (PS) stage), a subsequent shader stage may look for these system values to optimize processing in that stage.

Related topics

Graphics pipeline

Primitive topologies

Article • 10/20/2022

Direct3D supports several primitive topologies, which define how vertices are interpreted and rendered by the pipeline, such as point lists, line lists, and triangle strips.

Basic primitive topologies

The following basic primitive topologies (or primitive types) are supported:

- Point lists
- Line lists
- Line strips
- Triangle lists
- Triangle strips

For a visualization of each primitive type, see the diagram later in this topic in Winding direction and leading vertex positions.

The Input Assembler (IA) stage reads data from vertex and index buffers, assembles the data into these primitives, and then sends the data to the remaining pipeline stages.

Primitive adjacency

All Direct3D primitive types (except the point list) are available in two versions: one primitive type with adjacency and one primitive type without adjacency. Primitives with adjacency contain some of the surrounding vertices, while primitives without adjacency contain only the vertices of the target primitive. For example, the line list primitive has a corresponding line list primitive that includes adjacency.

Adjacent primitives are intended to provide more information about your geometry and are only visible through a geometry shader. Adjacency is useful for geometry shaders that use silhouette detection, shadow volume extrusion, and so on.

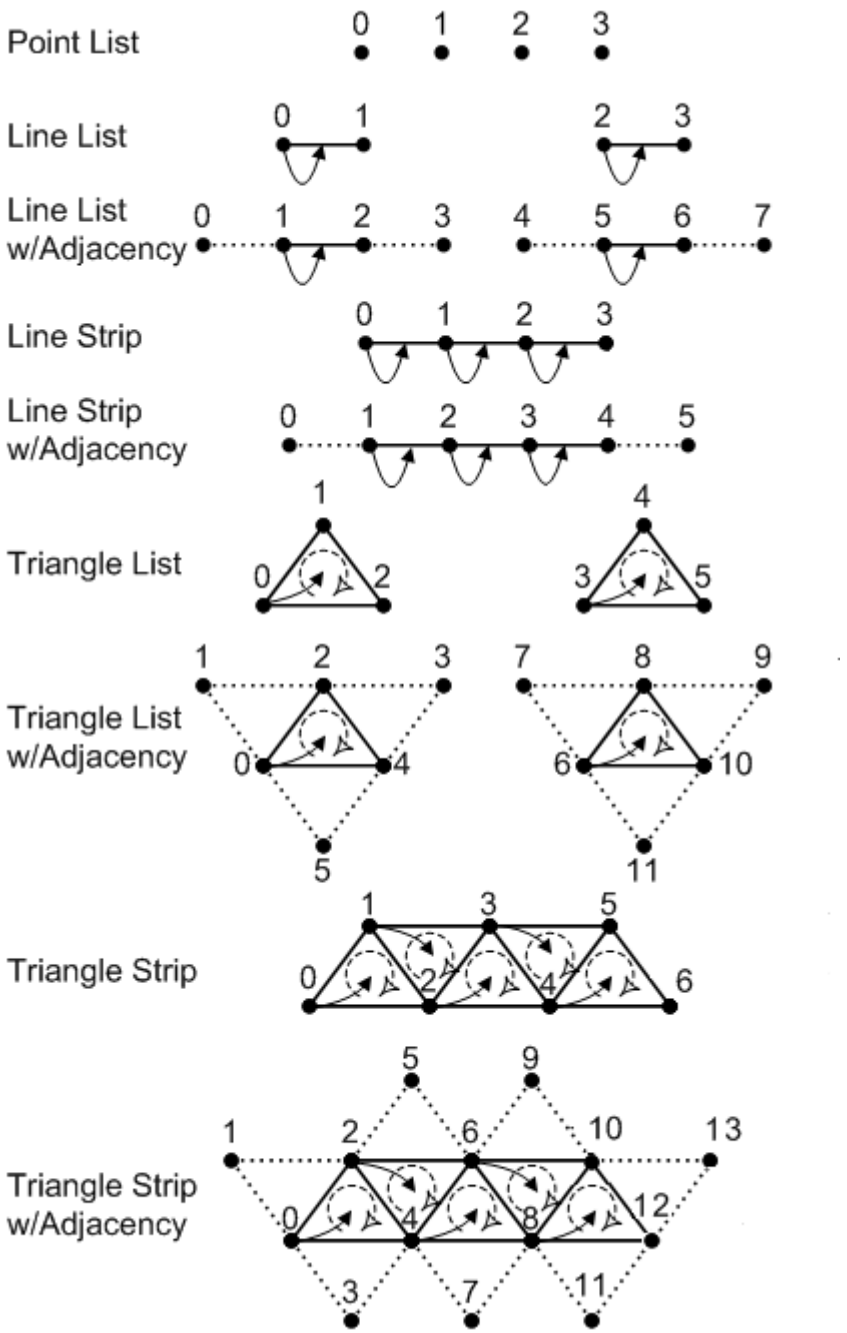
For example, suppose you want to draw a triangle list with adjacency. A triangle list that contains 36 vertices (with adjacency) will yield 6 completed primitives. Primitives with adjacency (except line strips) contain exactly twice as many vertices as the equivalent primitive without adjacency, where each additional vertex is an adjacent vertex.

Winding direction and leading vertex positions

As shown in the following illustration, a leading vertex is the first non-adjacent vertex in a primitive. A primitive type can have multiple leading vertices defined, as long as each one is used for a different primitive.

- For a triangle strip with adjacency, the leading vertices are 0, 2, 4, 6, and so on.
- For a line strip with adjacency, the leading vertices are 1, 2, 3, and so on.
- An adjacent primitive, on the other hand, has no leading vertex.

The following illustration shows the vertex ordering for all of the primitive types that the input assembler can produce.



The symbols in the preceding illustration are described in the following table.

Symbol	Name	Description

Symbol	Name	Description
	Vertex	A point in 3D space.
	Winding Direction	The vertex order when assembling a primitive. Can be clockwise or counterclockwise.
	Leading Vertex	The first non-adjacent vertex in a primitive that contains per-constant data.

Generating multiple strips

You can generate multiple strips through strip cutting. You can perform a strip cut by explicitly calling the RestartStrip HLSL function, or by inserting a special index value into the index buffer. This value is –1, which is 0xffffffff for 32-bit indices or 0xffff for 16-bit indices.

An index of –1 indicates an explicit 'cut' or 'restart' of the current strip. The previous index completes the previous primitive or strip and the next index starts a new primitive or strip.

For more info about generating multiple strips, see Geometry Shader (GS) stage.

Related topics

Input-Assembler (IA) stage

Graphics pipeline

Using system-generated values

Article • 10/20/2022

System-generated values are generated by the Input Assembler (IA) stage (based on user-supplied input semantics) to allow certain efficiencies in shader operations. By attaching data, such as an instance ID (visible to the Vertex Shader (VS) stage), a vertex ID (visible to VS), or a primitive ID (visible to Geometry Shader (GS) stage/Pixel Shader (PS) stage), a subsequent shader stage may look for these system values to optimize processing in that stage.

For instance, the VS stage may look for the instance ID to grab additional per-vertex data for the shader or to perform other operations; the GS and PS stages may use the primitive ID to grab per-primitive data in the same way.

VertexID

A vertex ID is used by each shader stage to identify each vertex. It is a 32-bit unsigned integer whose default value is 0. It is assigned to a vertex when the primitive is processed by the Input Assembler (IA) stage. Attach the vertex-ID semantic to the shader input declaration to inform the IA stage to generate a per-vertex ID.

The IA will add a vertex ID to each vertex for use by shader stages. For each draw call, the vertex ID is incremented by 1. Across indexed draw calls, the count resets back to the start value. If the vertex ID overflows (exceeds 2^{32}– 1), it wraps to 0.

For all primitive types, vertices have a vertex ID associated with them (regardless of adjacency).

PrimitiveID

A primitive ID is used by each shader stage to identify each primitive. It is a 32-bit unsigned integer whose default value is 0. It is assigned to a primitive when the primitive is processed by the Input Assembler (IA) stage. To inform the IA stage to generate a primitive ID, attach the primitive-ID semantic to the shader input declaration.

The IA stage will add a primitive ID to each primitive for use by the Geometry Shader (GS) stage or the Vertex Shader (VS) stage (whichever is the first stage active after the IA stage). For each indexed draw call, the primitive ID is incremented by 1, however, the primitive ID resets to 0 whenever a new instance begins. All other draw calls do not

change the value of the instance ID. If the instance ID overflows (exceeds $2^{32}-1$), it wraps to 0.

The Pixel Shader (PS) stage does not have a separate input for a primitive ID; however, any pixel shader input that specifies a primitive ID uses a constant interpolation mode.

There is no support for automatically generating a primitive ID for adjacent primitives. For primitive types with adjacency, such as a triangle strip with adjacency, a primitive ID is only maintained for the interior primitives (the non-adjacent primitives), just like the set of primitives in a triangle strip without adjacency.

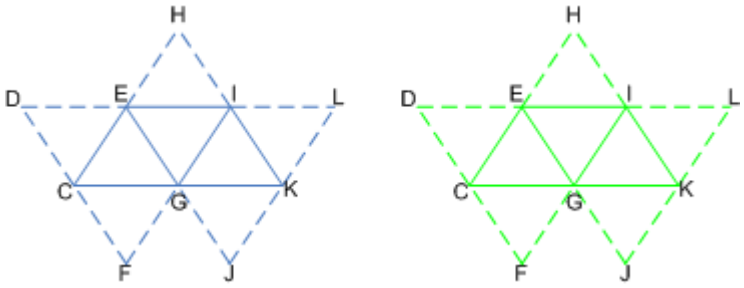
InstanceID

An instance ID is used by each shader stage to identify the instance of the geometry that is currently being processed. It is a 32-bit unsigned integer whose default value is 0.

The Input Assembler (IA) stage will add an instance ID to each vertex if the vertex shader input declaration includes the instance ID semantic. For each indexed draw call, instance ID is incremented by 1. All other draw calls do not change the value of instance ID. If instance ID overflows (exceeds $2^{32}-1$), it wraps to 0.

Example

The following illustration shows how system values are attached to an instanced triangle strip in the Input Assembler (IA) stage.



These tables show the system values generated for both instances of the same triangle strip. The first instance (instance U) is shown in blue, the second instance (instance V) is shown in green. The solid lines connect the vertices in the primitives, the dashed lines connect the adjacent vertices.

The following tables show the system-generated values for the instance U.

Vertex Data	C,U	D,U	E,U	F,U	G,U	H,U	I,U	J,U	K,U	L,U
VertexID	0	1	2	3	4	5	6	7	8	9

Vertex Data	C,U	D,U	E,U	F,U	G,U	H,U	I,U	J,U	K,U	L,U
InstanceID	0	0	0	0	0	0	0	0	0	0

Triangle strip instance U has 3 triangle primitives, with the following system-generated values:

	Value 1	Value 2	Value 3
PrimitiveID	0	1	2
InstanceID	0	0	0

The following tables show the system-generated values for the instance V.

Vertex Data	C,V	D,V	E,V	F,V	G,V	H,V	I,V	J,V	K,V	L,V
VertexID	0	1	2	3	4	5	6	7	8	9
InstanceID	1	1	1	1	1	1	1	1	1	1

Triangle strip instance V has 3 triangle primitives, with the following system-generated values:

	Value 1	Value 2	Value 3
PrimitiveID	0	1	2
InstanceID	1	1	1

The Input Assembler (IA) stage generates the IDs (vertex, primitive, and instance); notice also that each instance is given a unique instance ID. The data ends with the strip cut, which separates each instance of the triangle strip.

Related topics

Input-Assembler (IA) stage

Vertex Shader (VS) stage

Article • 10/20/2022

The Vertex Shader (VS) stage processes vertices, typically performing operations such as transformations, skinning, and lighting. A vertex shader takes a single input vertex and produces a single output vertex.

Purpose and uses

The Vertex Shader (VS) stage is used for individual per-vertex processing such as:

- Transformations
- Skinning
- Morphing
- Per-vertex lighting

The Vertex Shader stage is a programmable-shader stage; it is shown as a rounded block in the graphics pipeline diagram. This shader stage uses shader model 4.0 common-shader core.

The vertex-shader (VS) stage processes vertices from the input assembler. Vertex shaders always operate on a single input vertex and produce a single output vertex. The vertex shader stage must always be active for the pipeline to execute. If no vertex modification or transformation is required, a pass-through vertex shader must be created and set to the pipeline.

Each vertex shader input vertex can be comprised of up to 16 32-bit vectors (up to 4 components each). Each output vertex can be comprised of as many as 16 32-bit 4-component vectors. All vertex shaders must have a minimum of one input and one output, which can be as little as one scalar value.

The vertex-shader stage can consume two system-generated values from the input assembler: VertexID and InstanceID (see System Values and Semantics). Since VertexID and InstanceID are both meaningful at a vertex level, and IDs generated by hardware can only be fed into the first stage that understands them, these ID values can only be fed into the vertex-shader stage.

Vertex shaders are always run on all vertices, including adjacent vertices in input primitive topologies with adjacency. The number of times that the vertex shader has been executed can be queried from the CPU using the VSInvocations pipeline statistic.

A vertex shader can perform load and texture sampling operations where screen-space derivatives are not required (using HLSL intrinsic functions: Sample (DirectX HLSL Texture Object), SampleCmpLevelZero (DirectX HLSL Texture Object), and SampleGrad (DirectX HLSL Texture Object)).

Input

A single vertex, with VertexID and InstanceID system-generated values. Each vertex shader input vertex can be comprised of up to 16 32-bit vectors (up to 4 components each).

Output

A single vertex. Each output vertex can be comprised of as many as 16 32-bit 4-component vectors.

Related topics

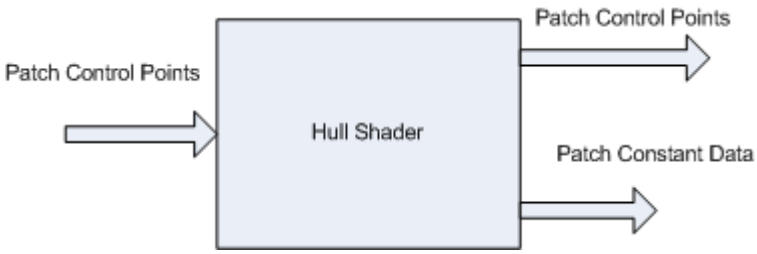
Graphics pipeline

Hull Shader (HS) stage

Article • 10/20/2022

The Hull Shader (HS) stage is one of the tessellation stages, which efficiently break up a single surface of a model into many triangles. The Hull Shader (HS) stage produces a geometry patch (and patch constants) that correspond to each input patch (quad, triangle, or line). A hull shader is invoked once per patch, and it transforms input control points that define a low-order surface into control points that make up a patch. It also does some per-patch calculations to provide data for the Tessellator (TS) stage and the Domain Shader (DS) stage.

Purpose and uses



The three tessellation stages work together to convert higher-order surfaces (which keep the model simple and efficient) to many triangles for detailed rendering within the graphics pipeline. The tessellation stages include the Hull Shader (HS) stage, Tessellator (TS) stage, and Domain Shader (DS) stage.

The Hull Shader (HS) stage is a programmable shader stage. A hull shader is implemented with an HLSL function.

A hull shader operates in two phases: a control-point phase and a patch-constant phase, which are run in parallel by the hardware. The HLSL compiler extracts the parallelism in a hull shader and encodes it into bytecode that drives the hardware.

- The control-point phase operates once for each control-point, reading the control points for a patch, and generating one output control point (identified by a **ControlPointID**).
- The patch-constant phase operates once per patch to generate edge tessellation factors and other per-patch constants. Internally, many patch-constant phases may run at the same time. The patch-constant phase has read-only access to all input and output control points.

Input

Between 1 and 32 input control points, which together define a low-order surface.

- The hull shader declares the state required by the Tessellator (TS) stage. This includes information such as the number of control points, the type of patch face and the type of partitioning to use when tessellating. This information appears as declarations typically at the front of the shader code.
- Tessellation factors determine how much to subdivide each patch.

Output

Between 1 and 32 output control points, which together make up a patch.

- The shader output is between 1 and 32 control points, regardless of the number of tessellation factors. The control-points output from a hull shader can be consumed by the domain-shader stage. Patch constant data can be consumed by a domain shader. Tessellation factors can be consumed by the Tessellator (TS) stage and the Domain Shader (DS) stage.
- If the hull shader sets any edge tessellation factor to ≤ 0 or NaN, the patch will be culled (omitted). As a result, the tessellator stage may or may not run, the domain shader will not run, and no visible output will be produced for that patch.

Example

hlsl

```
[patchsize(12)]
[patchconstantfunc(MyPatchConstantFunc)]
MyOutPoint main(uint Id : SV_ControlPointID,
     InputPatch<MyInPoint, 12> InPts)
{
     MyOutPoint result;

     ...

     result = TransformControlPoint( InPts[Id] );

     return result;
}
```

See How To: Create a Hull Shader.

Related topics

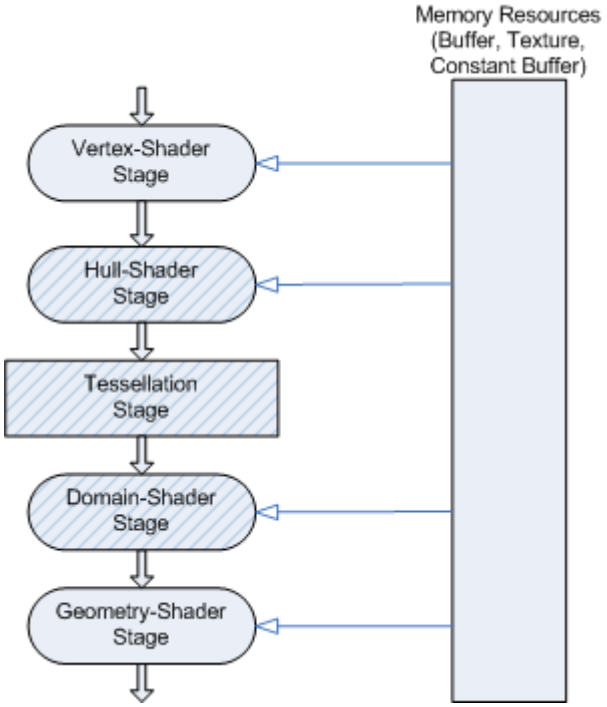
Tessellator (TS) stage

Article • 10/20/2022

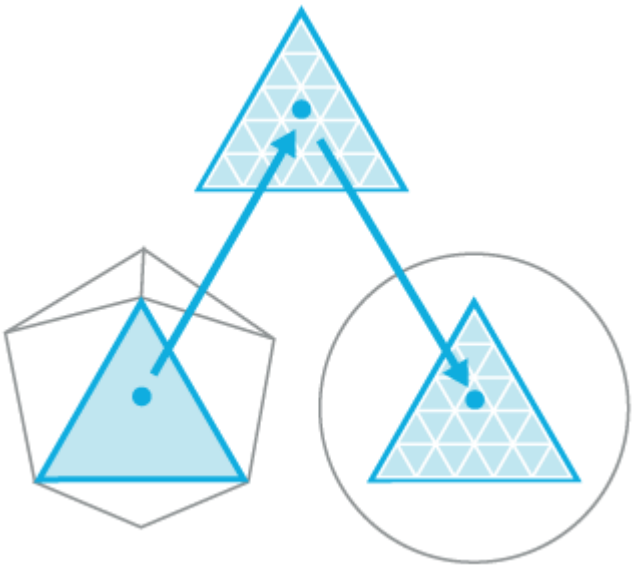
The Tessellator (TS) stage creates a sampling pattern of the domain that represents the geometry patch and generates a set of smaller objects (triangles, points, or lines) that connect these samples.

Purpose and uses

The following diagram highlights the stages of the Direct3D graphics pipeline.



The following diagram shows the progression through the tessellation stages.



The progression starts with the low-detail subdivision surface. The progression next highlights the input patch with the corresponding geometry patch, domain samples, and triangles that connect these samples. The progression finally highlights the vertices that correspond to these samples.

The Direct3D runtime supports three stages that implement tessellation, which converts low-detail subdivision surfaces into higher-detail primitives on the GPU. Tessellation tiles (or breaks up) high-order surfaces into suitable structures for rendering.

The tessellation stages work together to convert higher-order surfaces (which keep the model simple and efficient) to many triangles for detailed rendering within the Direct3D graphics pipeline.

Tessellation uses the GPU to calculate a more detailed surface from a surface constructed from quad patches, triangle patches or isolines. To approximate the high-ordered surface, each patch is subdivided into triangles, points, or lines using tessellation factors. The Direct3D graphics pipeline implements tessellation using three pipeline stages:

- Hull Shader (HS) stage - A programmable shader stage that produces a geometry patch (and patch constants) that correspond to each input patch (quad, triangle, or line).
- Tessellator (TS) stage - A fixed-function pipeline stage that creates a sampling pattern of the domain that represents the geometry patch and generates a set of smaller objects (triangles, points, or lines) that connect these samples.
- Domain Shader (DS) stage - A programmable shader stage that calculates the vertex position that corresponds to each domain sample.

By implementing tessellation in hardware, a graphics pipeline can evaluate lower detail (lower polygon count) models and render in higher detail. While software tessellation can be done, tessellation implemented by hardware can generate an incredible amount of visual detail (including support for displacement mapping) without adding the visual detail to the model sizes and paralyzing refresh rates.

Benefits of tessellation:

- Tessellation saves lots of memory and bandwidth, which allows an application to render higher detailed surfaces from low-resolution models. The tessellation technique implemented in the Direct3D graphics pipeline also supports displacement mapping, which can produce stunning amounts of surface detail.
- Tessellation supports scalable-rendering techniques, such as continuous or view dependent levels-of-detail which can be calculated on the fly.

- Tessellation improves performance by performing expensive computations at lower frequency (doing calculations on a lower-detail model). This could include blending calculations using blend shapes or morph targets for realistic animation or physics calculations for collision detection or soft body dynamics.

The Direct3D graphics pipeline implements tessellation in hardware, which off-loads the work from the CPU to the GPU. This can lead to very large performance improvements if an application implements large numbers of morph targets and/or more sophisticated skinning/deformation models.

The tessellator is a fixed-function stage initialized by binding a hull shader to the pipeline. (see How To: Initialize the Tessellator Stage). The purpose of the tessellator stage is to subdivide a domain (quad, tri, or line) into many smaller objects (triangles, points or lines). The tessellator tiles a canonical domain in a normalized (zero-to-one) coordinate system. For example, a quad domain is tessellated to a unit square.

Phases in the Tessellator (TS) stage

The Tessellator (TS) stage operates in two phases:

- The first phase processes the tessellation factors, fixing rounding problems, handling very small factors, reducing and combining factors, using 32-bit floating-point arithmetic.

- The second phase generates point or topology lists based on the type of partitioning selected. This is the core task of the tessellator stage and uses 16-bit fractions with fixed-point arithmetic. Fixed-point arithmetic allows hardware acceleration while maintaining acceptable precision. For example, given a 64 meter wide patch, this precision can place points at a 2 mm resolution.

Type of Partitioning	**Range**
Fractional_odd	[1...63]
Fractional_even	TessFactor range: [2..64]
Integer	TessFactor range: [1..64]
Pow2	TessFactor range: [1..64]

Tessellation is implemented with two programmable shader stages: a hull shader and a domain shader. These shader stages are programmed with HLSL code that is defined in shader model 5. The shader targets are: hs_5_0 and ds_5_0. The title creates the shader,

then code for the hardware is extracted from compiled shaders passed into the runtime when shaders are bound to the pipeline.

Enabling/disabling tessellation

Enable tessellation by creating a hull shader and binding it to the hull-shader stage (this automatically sets up the tessellator stage). To generate the final vertex positions from the tessellated patches, you will also need to create a domain shader and bind it to the domain-shader stage. Once tessellation is enabled, the data input to the Input Assembler (IA) stage must be patch data. The input assembler topology must be a patch constant topology.

To disable tessellation, set the hull shader and the domain shader to **NULL**. Neither the Geometry Shader (GS) stage nor the Stream Output (SO) stage can read hull-shader output-control points or patch data.

Input

The tessellator operates once per patch using the tessellation factors (which specify how finely the domain will be tessellated) and the type of partitioning (which specifies the algorithm used to slice up a patch) that are passed in from the hull-shader stage.

Output

The tessellator outputs uv (and optionally w) coordinates and the surface topology to the domain-shader stage.

Related topics

Graphics pipeline

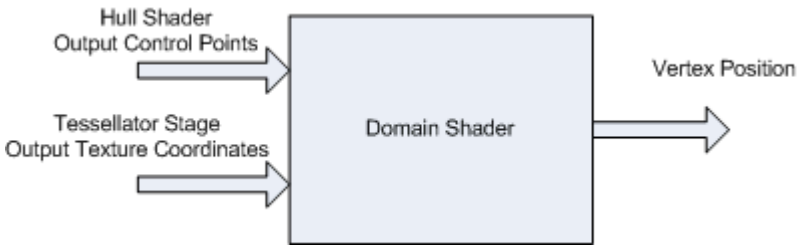
Domain Shader (DS) stage

Article • 10/20/2022

The Domain Shader (DS) stage calculates the vertex position of a subdivided point in the output patch; it calculates the vertex position that corresponds to each domain sample. A domain shader is run once per tessellator stage output point and has read-only access to the hull shader output patch and output patch constants, and the tessellator stage output UV coordinates.

Purpose and uses

The Domain Shader (DS) stage outputs the vertex position of a subdivided point in the output patch, based on input from the Hull Shader (HS) stage and the Tessellator (TS) stage.



Input

- A domain shader consumes output control points from the Hull Shader (HS) stage. The hull shader outputs include:
  - Control points.
  - Patch constant data.
  - Tessellation factors. The tessellation factors can include the values used by the fixed-function tessellator as well as the raw values (before rounding by integer tessellation, for example), which facilitates geomorphing, for example.
- A domain shader is invoked once per output coordinate from the Tessellator (TS) stage.

Output

- The Domain Shader (DS) stage outputs the vertex position of a subdivided point in the output patch.

After the domain shader completes, tessellation is finished and pipeline data continues to the next pipeline stage, such as the Geometry Shader (GS) stage and the Pixel Shader (PS) stage. A geometry shader that expects primitives with adjacency (for example, 6 vertices per triangle) is not valid when tessellation is active (this results in undefined behavior, which the debug layer will complain about).

Example

hlsl

```
void main( out    MyDSOutput result,
           float2 myInputUV : SV_DomainPoint,
           MyDSInput DSInputs,
           OutputPatch<MyOutPoint, 12> ControlPts,
           MyTessFactors tessFactors)
{
     ...

     result.Position = EvaluateSurfaceUV(ControlPoints, myInputUV);
}
```

Related topics

Graphics pipeline

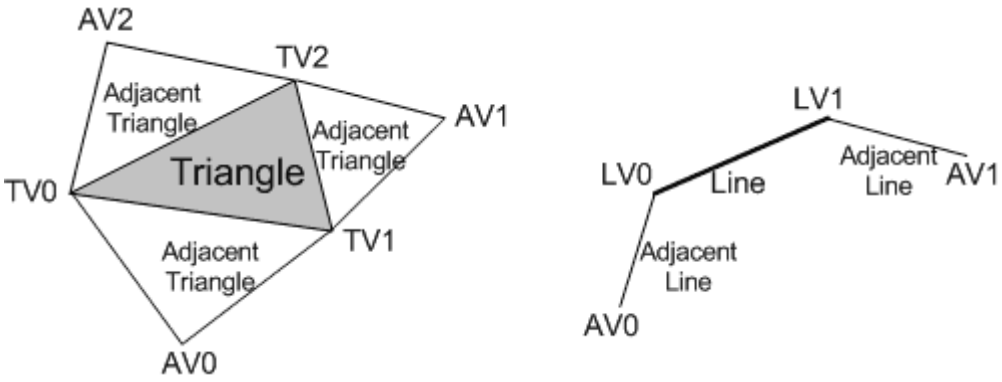
Geometry Shader (GS) stage

Article • 10/20/2022

The Geometry Shader (GS) stage processes entire primitives: triangles, lines, and points, along with their adjacent vertices. It is useful for algorithms including Point Sprite Expansion, Dynamic Particle Systems, and Shadow Volume Generation. It supports geometry amplification and de-amplification.

Purpose and uses

The Geometry Shader stage processes entire primitives: triangles (3 vertices with up to 3 adjacent vertices), lines (2 vertices with up to 2 adjacent vertices), and points (1 vertex).



The Geometry Shader also supports limited geometry amplification and de-amplification. Given an input primitive, the Geometry Shader can discard the primitive, or emit one or more new primitives.

The Geometry Shader (GS) stage is a programmable-shader stage; it is shown as a rounded block in the graphics pipeline diagram. This shader stage exposes its own unique functionality, built on the shader models (see common-shader core).

The Geometry Shader stage is well-suited for algorithms including:

- Point Sprite Expansion
- Dynamic Particle Systems
- Fur/Fin Generation
- Shadow Volume Generation
- Single Pass Render-to-Cubemap
- Per-Primitive Material Swapping
- Per-Primitive Material Setup - This capability includes generation of barycentric coordinates as primitive data so that a pixel shader can perform custom attribute interpolation.

Input

The Geometry Shader stage runs application-specified shader code with entire primitives as input and the ability to generate vertices on output. Unlike vertex shaders, which operate on a single vertex, the geometry shader's inputs are the vertices for a full primitive (three vertices for triangles, two vertices for lines, or single vertex for point). Geometry shaders can also bring in the vertex data for the edge-adjacent primitives as input (an additional three for a triangle, an additional two vertices for a line).

The Geometry Shader stage can consume the **SV_PrimitiveID** system-generated value that is auto-generated by the Input Assembler (IA) stage. This allows per-primitive data to be fetched or computed if desired.

When a geometry shader is active, it is invoked once for every primitive passed down or generated earlier in the pipeline. Each invocation of the geometry shader sees as input the data for the invoking primitive, whether that is a single point, a single line, or a single triangle. A triangle strip from earlier in the pipeline would result in an invocation of the geometry shader for each individual triangle in the strip (as if the strip were expanded out into a triangle list). All the input data for each vertex in the individual primitive is available (that is, 3 vertices for a triangle), plus adjacent vertex data if applicable and available.

Common vertex abbreviations:

Abbreviation	Term
TV	Triangle vertex
LV	Line vertex
AV	Adjacent vertex

Output

The Geometry Shader (GS) stage is capable of outputting multiple vertices forming a single selected topology. Available geometry shader output topologies are **tristrip**, **linestrip**, and **pointlist**. The number of primitives emitted can vary freely within any invocation of the geometry shader, though the maximum number of vertices that could be emitted must be declared statically. Strip lengths emitted from a geometry shader invocation can be arbitrary, and new strips can be created via the RestartStrip HLSL function.

Execution of a geometry shader instance is atomic from other invocations, except that data added to the streams is serial. The outputs of a given invocation of a geometry shader are independent of other invocations (though ordering is respected). A geometry shader generating triangle strips will start a new strip on every invocation.

Geometry shader output may be fed to the rasterizer stage and/or to a vertex buffer in memory via the stream output stage. Output fed to memory is expanded to individual point/line/triangle lists (exactly as they would be passed to the rasterizer).

A geometry shader outputs data one vertex at a time by appending vertices to an output stream object. The topology of the streams is determined by a fixed declaration, choosing a **TriangleStream**, **LineStream** and **PointStream** as the output for the GS stage.

There are three types of stream objects available: **TriangleStream**, **LineStream** and **PointStream**, which are all templated objects. The topology of the output is determined by their respective object type, while the format of the vertices appended to the stream is determined by the template type.

When a geometry shader output is identified as a System Interpreted Value (for example, **SV_RenderTargetArrayIndex** or **SV_Position**), hardware looks at this data and performs some behavior dependent on the value, in addition to being able to pass the data itself to the next shader stage for input. When such data output from the geometry shader has meaning to the hardware on a per-primitive basis (such as **SV_RenderTargetArrayIndex** or **SV_ViewportArrayIndex**), rather than on a per-vertex basis (such as **SV_ClipDistance[n]** or **SV_Position**), the per-primitive data is taken from the leading vertex emitted for the primitive.

Partially completed primitives could be generated by the geometry shader if the geometry shader ends and the primitive is incomplete. Incomplete primitives are silently discarded. This is similar to the way the IA treats partially completed primitives.

The geometry shader can perform load and texture sampling operations where screen-space derivatives are not required (**samplelevel**, **samplecmplevelzero**, **samplegrad**).

Related topics

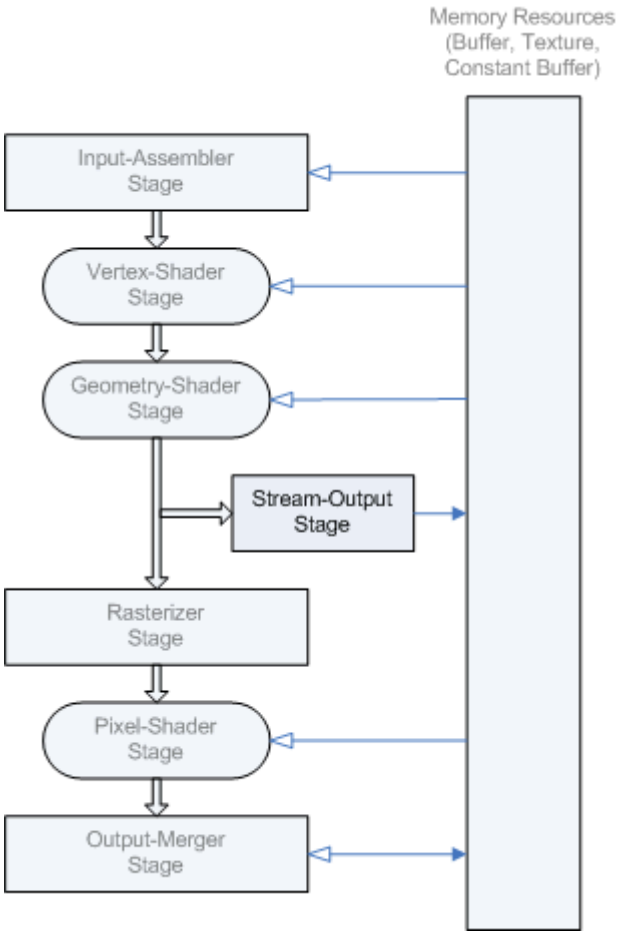
Graphics pipeline

Stream Output (SO) stage

Article • 10/20/2022

The Stream Output (SO) stage continuously outputs (or streams) vertex data from the previous active stage to one or more buffers in memory. Data streamed out to memory can be re-circulated back into the pipeline as input data, or read-back from the CPU.

Purpose and uses



The stream-output stage streams primitive data from the pipeline to memory on its way to the rasterizer. Data from the previous stage can be streamed out to memory, and/or passed into the rasterizer. Data streamed out to memory can be re-circulated back into the pipeline as input data, or read-back from the CPU.

Data streamed out to memory can be read back into the pipeline in a subsequent rendering pass, or can be copied to a staging resource (so it can be read by the CPU). The amount of data streamed out can vary; Direct3D is designed to handle the data without the need to query (the GPU) about the amount of data written.-->

There are two ways to feed stream-output data into the pipeline:

- Stream-output data can be fed back into the Input Assembler (IA) stage.
- Stream-output data can be read by programmable shaders using Load functions.

Input

Vertex data from a previous shader stage.

Output

The Stream Output (SO) stage continuously outputs (or streams) vertex data from the previous active stage, such as the Geometry Shader (GS) stage, to one or more buffers in memory. If the Geometry Shader (GS) stage is inactive, the Stream Output (SO) stage continuously outputs vertex data from the Domain Shader (DS) stage to buffers in memory (or if DS is also inactive, from the Vertex Shader (VS) stage).

When a triangle or line strip is bound to the Input Assembler (IA) stage, each strip is converted into a list before they are streamed out. Vertices are always written out as complete primitives (for example, 3 vertices at a time for triangles); incomplete primitives are never streamed out. Primitive types with adjacency discard the adjacency data before streaming data out.

The stream-output stage supports up to 4 buffers simultaneously.

- If you are streaming data into multiple buffers, each buffer can only capture a single element (up to 4 components) of per-vertex data, with an implied data stride equal to the element width in each buffer (compatible with the way single element buffers can be bound for input into shader stages). Furthermore, if the buffers have different sizes, writing stops as soon as any one of the buffers is full.
- If you are streaming data into a single buffer, the buffer can capture up to 64 scalar components of per-vertex data (256 bytes or less) or the vertex stride can be up to 2048 bytes.

Related topics

Graphics pipeline

Rasterizer (RS) stage

Article • 10/20/2022

The rasterizer clips primitives that aren't in view, prepares primitives for the Pixel Shader (PS) stage, and determines how to invoke pixel shaders. The rasterization stage converts vector information (composed of shapes or primitives) into a raster image (composed of pixels) for the purpose of displaying real-time 3D graphics.

Purpose and uses

During rasterization, each primitive is converted into pixels, while interpolating per-vertex values across each primitive. Rasterization includes clipping vertices to the view frustum, performing a divide by z to provide perspective, mapping primitives to a 2D viewport, and determining how to invoke the pixel shader. While using a pixel shader is optional, the rasterizer stage always performs clipping, a perspective divide to transform the points into homogeneous space, and maps the vertices to the viewport.

You may disable rasterization by telling the pipeline there is no pixel shader (set the Pixel Shader (PS) stage to **NULL** and disable depth and stencil testing). While disabled, rasterization-related pipeline counters will not update.

On hardware that implements hierarchical Z-buffer optimizations, you may enable preloading the z-buffer by setting the Pixel Shader (PS) stage to **NULL** while enabling depth and stencil testing.

See Rasterization rules.

Input

Vertices (x,y,z,w), coming into the Rasterizer stage are assumed to be in homogeneous clip-space. In this coordinate space the X axis points right, Y points up and Z points away from camera.

The fixed-function Rasterizer (RS) stage is fed by the Stream Output (SO) stage and/or by the previous pipeline stage, such as the Geometry Shader (GS) stage. If GS isn't used, RS is fed by the Domain Shader (DS) stage. If DS also isn't used, RS is fed by the Vertex Shader (VS) stage.

Output

Using the Pixel Shader (PS) stage is optional; the rasterizer stage can output directly to the Output Merger (OM) stage instead.

Related topics

Rasterization rules

Graphics pipeline

Pixel Shader (PS) stage

Article • 10/20/2022

The Pixel Shader (PS) stage receives interpolated data for a primitive, and generates per-pixel data such as color.

This is a programmable shader stage; it is shown as a rounded block in the graphics pipeline diagram. This shader stage exposes its own unique functionality, built on the shader model 4.0 common-shader core.

The Pixel Shader (PS) stage enables rich shading techniques such as per-pixel lighting and post-processing. A pixel shader is a program that combines constant variables, texture data, interpolated per-vertex values, and other data to produce per-pixel outputs. The Rasterizer (RS) stage invokes a pixel shader once for each pixel covered by a primitive, however, it is possible to specify a **NULL** shader to avoid running a shader.

When multisampling a texture, a pixel shader is invoked once per-covered pixel while a depth/stencil test occurs for each covered multisample. Samples that pass the depth/stencil test are updated with the pixel shader output color.

The pixel shader intrinsic functions produce or use derivatives of quantities with respect to screen space x and y. The most common use for derivatives is to compute level-of-detail calculations for texture sampling and in the case of anisotropic filtering, selecting samples along the axis of anisotropy. Typically, a hardware implementation runs a pixel shader on multiple pixels (for example a 2x2 grid) simultaneously, so that derivatives of quantities computed in the pixel shader can be reasonably approximated as deltas of the values at the same point of execution in adjacent pixels.

Inputs

When the pipeline is configured without a geometry shader, a pixel shader is limited to 16, 32-bit, 4-component inputs. Otherwise, a pixel shader can take up to 32, 32-bit, 4-component inputs.

Pixel shader input data includes vertex attributes (that can be interpolated with or without perspective correction) or can be treated as per-primitive constants. Pixel shader inputs are interpolated from the vertex attributes of the primitive being rasterized, based on the interpolation mode declared. If a primitive gets clipped before rasterization, the interpolation mode is honored during the clipping process as well.

Vertex attributes are interpolated (or evaluated) at pixel shader center locations. Pixel shader attribute interpolation modes are declared in an input register declaration, on a per-element basis in either an argument or an input structure. Attributes can be interpolated linearly, or with centroid sampling. See the section "Centroid Sampling of Attributes when Multisample Antialiasing" in Rasterization rules. Centroid evaluation is relevant only during multisampling to cover cases where a pixel is covered by a primitive but a pixel center may not be; centroid evaluation occurs as close as possible to the (non-covered) pixel center.

Inputs may also be declared with a system-value semantic, which marks a parameter that is consumed by other pipeline stages. For instance, a pixel position should be marked with the SV_Position semantic. The Input Assembler (IA) stage can produce one scalar for a pixel shader (using SV_PrimitiveID); the Rasterizer (RS) stage can also generate one scalar for a pixel shader (using SV_IsFrontFace).

Outputs

A pixel shader can output up to 8, 32-bit, 4-component colors, or no color if the pixel is discarded. Pixel shader output register components must be declared before they can be used; each register is allowed a distinct output-write mask.

Use the depth-write-enable state (in the Output Merger (OM) stage) to control whether depth data gets written to a depth buffer (or use the discard instruction to discard data for that pixel). A pixel shader can also output an optional 32-bit, 1-component, floating-point, depth value for depth testing (using the SV_Depth semantic). The depth value is output in the oDepth register, and replaces the interpolated depth value for depth testing (assuming depth testing is enabled). There is no way to dynamically change between using fixed-function depth or shader oDepth.

A pixel shader cannot output a stencil value.

Related topics

Graphics pipeline

Output Merger (OM) stage

Article • 10/20/2022

The Output Merger (OM) stage combines various types of output data (pixel shader values, depth and stencil information) with the contents of the render target and depth/stencil buffers to generate the final pipeline result.

Purpose and uses

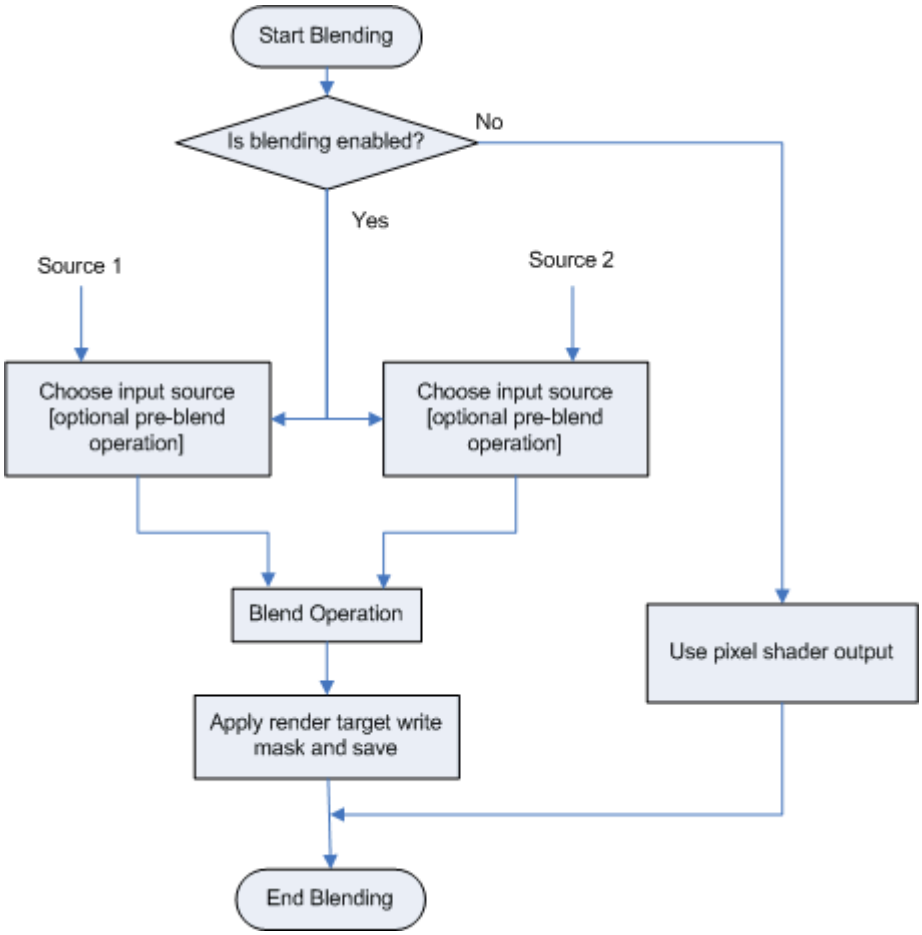
The Output Merger (OM) stage is the final step for determining which pixels are visible (with depth-stencil testing) and blending the final pixel colors.

The OM stage generates the final rendered pixel color using a combination of the following:

- Pipeline state
- The pixel data generated by the pixel shaders
- The contents of the render targets
- The contents of the depth/stencil buffers.

Blending overview

Blending combines one or more pixel values to create a final pixel color. The following diagram shows the process involved in blending pixel data.



Conceptually, you can visualize this flow chart implemented twice in the Output Merger stage: the first one blends RGB data, while in parallel, a second one blends alpha data. To see how to use the API to create and set blend state, see Configuring Blending Functionality.

Fixed-function blend can be enabled independently for each render target. However there is only one set of blend controls, so that the same blend is applied to all RenderTargets with blending enabled. Blend values (including BlendFactor) are always clamped to the range of the render-target format before blending. Clamping is done per render target, respecting the render target type. The only exception is for the float16, float11 or float10 formats which are not clamped so that blend operations on these formats can be done with at least equal precision/range as the output format. NaN's and signed zeros are propagated for all cases (including 0.0 blend weights).

When you use sRGB render targets, the runtime converts the render target color into linear space before it performs blending. The runtime converts the final blended value back into sRGB space before it saves the value back to the render target.

Dual-source color blending

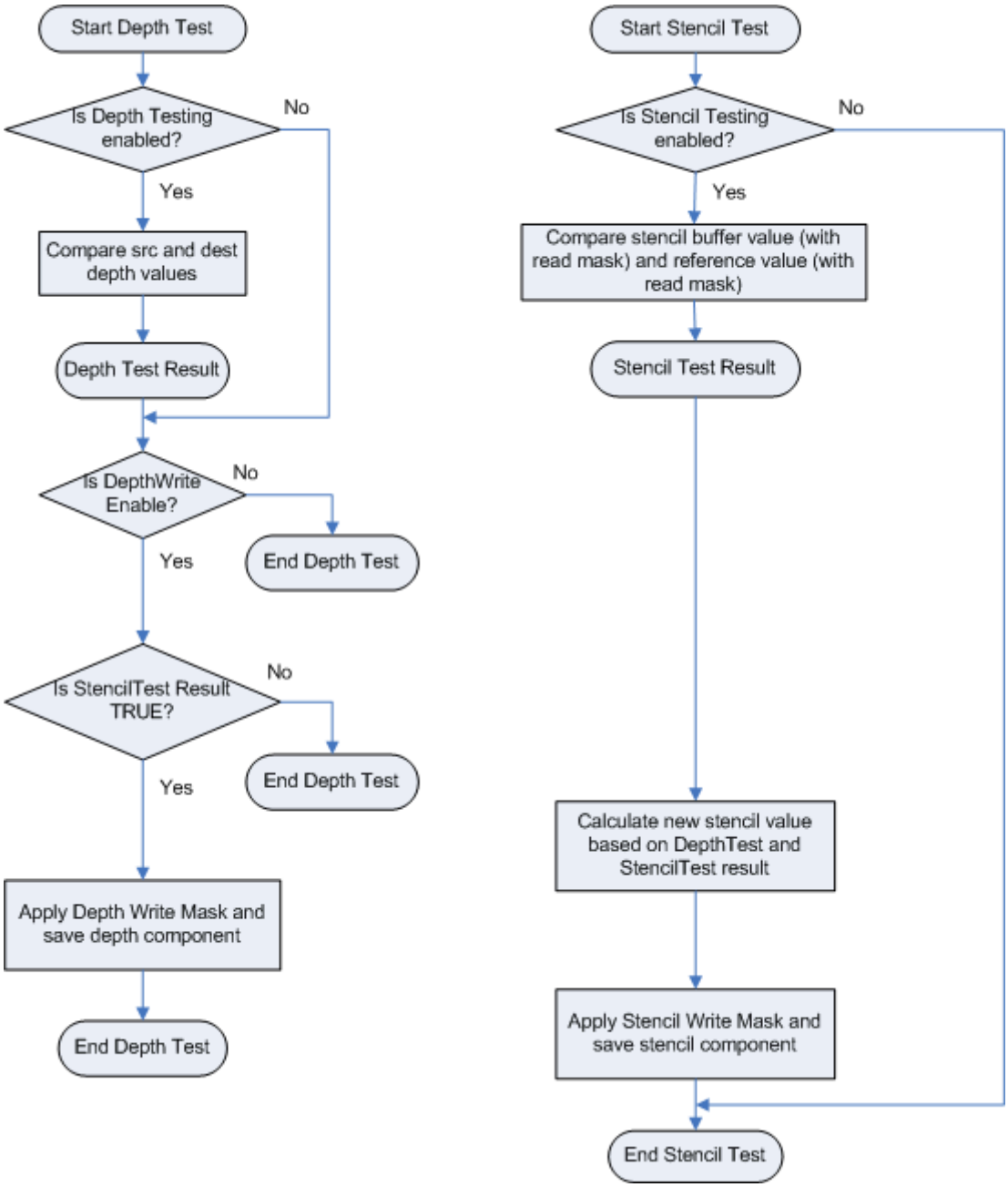
This feature enables the Output Merger stage to simultaneously use both pixel shader outputs (o0 and o1) as inputs to a blending operation with the single render target at

slot 0. Valid blend operations include: add, subtract and revsubtract. The blend equation and the output write mask specify which components the pixel shader is outputting. Extra components are ignored.

Writing to other pixel shader outputs (o2, o3 etc.) is undefined; you may not write to a render target if it is not bound to slot 0. Writing oDepth is valid during dual source color blending.

Depth-stencil testing overview

A depth-stencil buffer, which is created as a texture resource, can contain both depth data and stencil data. The depth data is used to determine which pixels lie closest to the camera, and the stencil data is used to mask which pixels can be updated. Ultimately, both the depth and stencil values data are used by the output-merger stage to determine if a pixel should be drawn or not. The following diagram shows conceptually how depth-stencil testing is done.



To configure depth-stencil testing, see Configuring Depth-Stencil Functionality. A depth-stencil object encapsulates depth-stencil state. An application can specify depth-stencil state, or the OM stage will use default values. Blending operations are performed on a per-pixel basis if multisampling is disabled. If multisampling is enabled, blending occurs on a per-multisample basis.

The process of using the depth buffer to determine which pixel should be drawn is called depth buffering, also sometimes called z-buffering.

Once depth values reach the output-merger stage (whether coming from interpolation or from a pixel shader) they are always clamped: z = min(Viewport.MaxDepth,max(Viewport.MinDepth,z)) according to the format/precision of the depth buffer, using floating-point rules. After clamping, the depth value is compared (using DepthFunc) against the existing depth-buffer value. If no depth buffer is bound, the depth test always passes.

If there is no stencil component in the depth-buffer format, or no depth buffer bound, then the stencil test always passes.

Only one depth/stencil buffer can be active at a time; any bound resource view must match (same size and dimensions) the depth/stencil view. This does not mean the resource size must match, just that the view size must match.

Sample mask overview

A sample mask is a 32-bit multisample coverage mask that determines which samples get updated in active render targets. Only one sample mask is allowed. The mapping of bits in a sample mask to the samples in a resource is defined by a user. For n-sample rendering, the first n bits (from the LSB) of the sample mask are used (32 bits it the maximum number of bits).

Input

The Output Merger (OM) stage generates the final rendered pixel color using a combination of the following:

- Pipeline state
- The pixel data generated by the pixel shaders
- The contents of the render targets
- The contents of the depth/stencil buffers.

Output

Output-write mask overview

Use an output-write mask to control (per component) what data can be written to a render target.

Multiple render targets overview

A pixel shader can be used to render to at least 8 separate render targets, all of which must be the same type (buffer, Texture1D, Texture1DArray, and so on). Furthermore, all render targets must have the same size in all dimensions (width, height, depth, array size, sample counts). Each render target may have a different data format.

You may use any combination of render targets slots (up to 8). However, a resource view cannot be bound to multiple render-target-slots simultaneously. A view may be reused as long as the resources are not used simultaneously.

In this section

Topic	Description
Configuring depth-stencil functionality	This section covers the steps for setting up the depth-stencil buffer, and depth-stencil state for the output-merger stage.

Related topics

Graphics pipeline

Configuring depth-stencil functionality

Article • 10/20/2022

This section covers the steps for setting up the depth-stencil buffer, and depth-stencil state for the output-merger stage.

Once you know how to use the depth-stencil buffer and the corresponding depth-stencil state, refer to advanced-stencil techniques.

Create Depth-Stencil State

The depth-stencil state tells the output-merger stage how to perform the depth-stencil test. The depth-stencil test determines whether or not a given pixel should be drawn.

Bind Depth-Stencil Data to the OM Stage

Bind the depth-stencil state.

Bind the depth-stencil resource using a view.

Render targets must all be the same type of resource. If multisample antialiasing is used, all bound render targets and depth buffers must have the same sample counts.

When a buffer is used as a render target, depth-stencil testing and multiple render targets are not supported.

- As many as 8 render targets can be bound simultaneously.
- All render targets must have the same size in all dimensions (width and height, and depth for 3D or array size for *Array types).
- Each render target may have a different data format.
- Write masks control what data gets written to a render target. The output write masks control on a per-render target, per-component level what data gets written to the render target(s).

Advanced Stencil Techniques

The stencil portion of the depth-stencil buffer can be used for creating rendering effects such as compositing, decaling, and outlining.

- Compositing
- Decaling

- Outlines and Silhouettes
- Two-Sided Stencil
- Reading the Depth-Stencil Buffer as a Texture

Compositing

Your application can use the stencil buffer to composite 2D or 3D images onto a 3D scene. A mask in the stencil buffer is used to occlude an area of the rendering target surface. Stored 2D information, such as text or bitmaps, can then be written to the occluded area. Alternately, your application can render additional 3D primitives to the stencil-masked region of the rendering target surface. It can even render an entire scene.

Games often composite multiple 3D scenes together. For instance, driving games typically display a rear-view mirror. The mirror contains the view of the 3D scene behind the driver. It is essentially a second 3D scene composited with the driver's forward view.

Decaling

Direct3D applications use decaling to control which pixels from a particular primitive image are drawn to the rendering target surface. Applications apply decals to the images of primitives to enable coplanar polygons to render correctly.

For instance, when applying tire marks and yellow lines to a roadway, the markings should appear directly on top of the road. However, the z values of the markings and the road are the same. Therefore, the depth buffer might not produce a clean separation between the two. Some pixels in the back primitive may be rendered on top of the front primitive and vice versa. The resulting image appears to shimmer from frame to frame. This effect is called z-fighting or flimmering.

To solve this problem, use a stencil to mask the section of the back primitive where the decal will appear. Turn off z-buffering and render the image of the front primitive into the masked-off area of the render-target surface.

Multiple texture blending can be used to solve this problem.

Outlines and Silhouettes

You can use the stencil buffer for more abstract effects, such as outlining and silhouetting.

If your application does two render passes - one to generate the stencil mask and second to apply the stencil mask to the image, but with the primitives slightly smaller on the second pass - the resulting image will contain only the primitive's outline. The application can then fill the stencil-masked area of the image with a solid color, giving the primitive an embossed look.

If the stencil mask is the same size and shape as the primitive you are rendering, the resulting image contains a hole where the primitive should be. Your application can then fill the hole with black to produce a silhouette of the primitive.

Two-Sided Stencil

Shadow Volumes are used for drawing shadows with the stencil buffer. The application computes the shadow volumes cast by occluding geometry, by computing the silhouette edges and extruding them away from the light into a set of 3D volumes. These volumes are then rendered twice into the stencil buffer.

The first render draws forward-facing polygons, and increments the stencil-buffer values. The second render draws the back-facing polygons of the shadow volume, and decrements the stencil buffer values.

Normally, all incremented and decremented values cancel each other out. However, the scene was already rendered with normal geometry causing some pixels to fail the z-buffer test as the shadow volume is rendered. Values left in the stencil buffer correspond to pixels that are in the shadow. These remaining stencil-buffer contents are used as a mask, to alpha-blend a large, all-encompassing black quad into the scene. With the stencil buffer acting as a mask, the result is to darken pixels that are in the shadows.

This means that the shadow geometry is drawn twice per light source, hence putting pressure on the vertex throughput of the GPU. The two-sided stencil feature has been designed to mitigate this situation. In this approach, there are two sets of stencil state (named below), one set each for the front-facing triangles and the other for the back-facing triangles. This way, only a single pass is drawn per shadow volume, per light.

Reading the Depth-Stencil Buffer as a Texture

An inactive depth-stencil buffer can be read by a shader as a texture. An application that reads a depth-stencil buffer as a texture renders in two passes, the first pass writes to the depth-stencil buffer and the second pass reads from the buffer. This allows a shader to compare depth or stencil values previously written to the buffer against the value for the pixel currently being rendered. The result of the comparison can be used to create effects such as shadow mapping or soft particles in a particle system.

Related topics

Output Merger (OM) stage

Graphics pipeline

Output-Merger Stage

Feedback

Was this page helpful? Yes No

Provide product feedback | Get help at Microsoft Q&A

State objects

Article • 10/20/2022

Device state is grouped into state objects which greatly reduce the cost of state changes. There are several state objects, and each one is designed to initialize a set of state for a particular pipeline stage. State objects vary by version of Direct3D.

Input-Layout State

This group of state dictates how the Input Assembler (IA) stage reads data out of the input buffers and assembles it for use by the vertex shader. This includes state such as the number of elements in the input buffer and the signature of the input data. The Input Assembler (IA) stage streams primitives from memory into the pipeline.

Rasterizer State

This group of state initializes the Rasterizer (RS) stage. This object includes state such as fill or cull modes, enabling a scissor rectangle for clipping, and setting multisample parameters. This stage rasterizes primitives into pixels, performing operations like clipping and mapping primitives to the viewport.

Depth-Stencil State

This group of state initializes the depth-stencil portion of the Output Merger (OM) stage. More specifically, this object initializes depth and stencil testing.

Blend State

This group of state initializes the blending portion of the Output Merger (OM) stage.

Sampler State

This group of state initializes a sampler object. A sampler object is used by the shader stages to filter textures in memory.

In Direct3D, the sampler object is not bound to a specific texture, it just describes how to do filtering given any attached resource.